Deep Q-Networks for Dots & Boxes

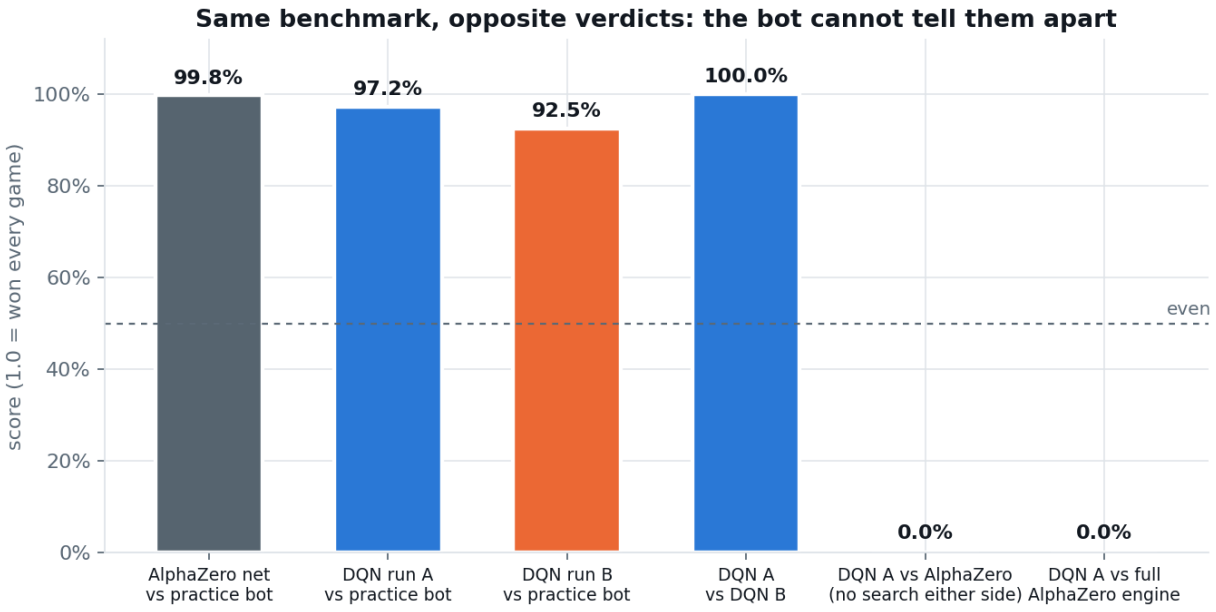
A complete account of the code, the approach, the settings, and what the experiment actually showed.

Board	5 x 5 boxes (25 boxes, 60 lines, 6 x 6 dots)
Algorithm	Dueling Double DQN, n-step returns, self-play
Hardware	2 x NVIDIA Tesla T4 (Kaggle), one run per GPU
Training	8.46 hours wall clock, two configurations in parallel
Run A total	2,047,872 games | 122,876,928 moves | 319,864 gradient steps
Run B total	1,482,240 games | 88,949,760 moves | 347,268 gradient steps
Date	18-19 September 2026

The headline

The DQN works. It learned to play Dots & Boxes well from nothing but self-play and the final score, and it beats the fixed benchmark bot in 97% of games. It also lost **every single one** of 200 games against the AlphaZero network from the earlier experiment, with search switched off on both sides, by an average of nine boxes.

> ***In plain words.*** *The DQN is not broken and it is not weak. It is simply beaten by a better method on the same game, and the margin is not close.*



Every matchup run for this report. The first three bars are all against the practice bot, where DQN and AlphaZero look almost identical. The last two are what happens when they play each other.

1. What we set out to do, and why

The project already had a strong Dots & Boxes engine built on AlphaZero: a neural network guiding a Monte-Carlo tree search, with an exact alpha-beta solver taking over in the endgame. The open question was whether a *value-based* method - the DQN family you work with on drone control - could reach the same level on the same game.

This is not an idle question. The two families make opposite trades. AlphaZero spends compute at decision time: it simulates hundreds of continuations before every move. DQN spends nothing at decision time: it looks at the board once and picks. If DQN could get close, it would be the cheaper engine by a wide margin.

> ***In plain words.*** *AlphaZero thinks before it moves. DQN just looks and moves. We wanted to know how much that thinking is actually worth, measured rather than argued.*

The rule that decides the design

Dots & Boxes has one rule that breaks almost every off-the-shelf implementation: **completing a box gives you another turn**. The player does not alternate reliably. A move can leave the board with the same player to move, or with the opponent to move, depending on whether it closed a box. Every part of the code below has to respect that, and getting it wrong makes the agent learn the opposite of what it should about half the time. This is the single most important correctness issue in the whole project.

2. The game as a reinforcement learning problem

State. The board: which of the 60 lines are drawn, which of the 25 boxes are owned and by whom, the score, and whose turn it is.

Action. Which undrawn line to draw. Up to 60 choices, shrinking as the game fills up. Illegal actions are masked to minus infinity everywhere they could be selected, so the agent can never pick an already-drawn line.

Reward. This is the design decision that matters most, so it gets its own section below.

Episode. Exactly 60 moves, always. The game cannot end early and cannot go long.

2.1 Reward: paying early without cheating

The obvious reward is the one the game gives you: nothing for 59 moves, then the final margin. That is the worst case for a value-based method. The information about a mistake made on move 12 has to travel backwards through 48 links of a bootstrap chain, and every link adds approximation error.

So we pay immediately instead. Every move earns the player who made it:

```
r = (boxes captured by this move) / 25
```

The important property is that this is **not** a heuristic or a guess. Sum those small payments across a whole game, flipping the sign every time the turn actually changes, and they add up to *exactly* the final margin divided by 25. Nothing is invented and nothing is distorted - the same total is simply paid earlier.

> ***In plain words.*** *Instead of paying the worker once at the end of the month, we pay them daily. The total at the end of the month is identical to the rupee. But now they can tell which day's work was good.*

This is verified by a test that runs before every training session. It plays 32 random games, accumulates the signed rewards, and asserts the total equals the true final margin. If the shaping ever introduced bias, training would refuse to start.

```
PASS signed rewards telescope exactly to the final margin (no shaping bias)
```

2.2 What Q means here

Q(s, a) is the expected **remaining** margin for the player to move in state s, divided by 25 so it sits in the range -1 to +1. A Q of +0.4 means "from here, expect to finish about 10 boxes ahead".

3. The algorithm, piece by piece

3.1 Double DQN

A plain DQN builds its learning target with a maximum over the next state's actions, and a maximum over noisy estimates is systematically too optimistic. Double DQN splits the job: the **online** network picks which action is best, the **target** network says how good it is. Two different noisy estimates rarely err in the same direction, so most of the optimism cancels.

```
a* = argmax Q_online(s', a) y = r + sign * gamma^k * Q_target(s', a*)
```

> ***In plain words.*** *One person nominates the best option, a second person scores it. If you let the same person do both, they talk themselves into their own favourite.*

3.2 Dueling heads

The network does not output 60 independent numbers. It outputs one value for the position as a whole, V(s), and 60 advantages, A(s,a), then recombines them:

```
Q(s,a) = V(s) + A(s,a) - mean over LEGAL actions of A(s, .)
```

Most Dots & Boxes positions are dominated by whether the position itself is good, not by which of 40 safe lines you pick. Splitting the two lets the network learn "this position is losing" once, rather than 60 times. Note the mean is taken over legal moves only - including masked moves would drag the baseline around as the board fills.

3.3 N-step returns, and the sign that makes them correct

Instead of learning one move at a time, each training example spans several moves. Run A uses three, run B uses five. Fewer bootstrap links means less accumulated approximation error.

This is where the extra-turn rule bites. A normal n-step return assumes the turn alternates, so it can flip signs on a fixed schedule. Here it cannot. Our implementation walks the actual trajectory and flips the sign only on moves that did **not** capture a box:

```
sign = +1 for each move i in the window: r_acc += sign * gamma^i * r_i if not
captured_a_box(i): sign = -sign # the turn really did change
```

Each stored transition therefore carries its own sign telling the learner whether the state it bootstraps from belongs to the same player or the opponent. Getting this backwards is silent - the loss still goes down - so it has a dedicated unit test that walks a four-move sequence by hand and checks the sign and the accumulated reward.

```
PASS n-step returns carry the correct sign through the extra-turn rule
```

3.4 Self-play

Both sides of every game are the same network acting epsilon-greedily. There is no fixed opponent to overfit against, and the opponent improves exactly as fast as the agent does. 384 games run in parallel in lockstep, which is what keeps a T4 busy: one network forward pass decides a move for all 384 games at once.

3.5 What the network sees

Twenty planes on an 11 x 11 grid. The first sixteen are the obvious things - which lines are drawn, how many sides each box has, who owns what, the score difference, how many lines are left, whether a line would give a box away. The last four are the ones that matter most:

Plane	Meaning
16	size of the chain this box belongs to, divided by 25
17	1 if that chain is fully formed (every box already has exactly 2 sides)
18	number of long chains on the board, divided by 25
19	PARITY of that long-chain count (0 or 1)

Chain structure and its parity decide Dots & Boxes, and a small convolutional network cannot work them out for itself - counting connected regions is close to the worst case for a 3x3 kernel. So we compute the chain decomposition directly with a breadth-first search and hand it to the network as input. The DQN was given this help from the start; it is not handicapped relative to the AlphaZero engine.

> ***In plain words.*** *We do not ask the network to discover that four long chains is different from five. We tell it, on every move.*

4. The code

Five files, about 1,400 lines, no external RL library. Everything was written from scratch because no off-the-shelf implementation handles a two-player game whose turn order depends on the move.

File	Lines	What it does
game.py	~400	The board. Geometry, the 8 symmetries, move application, the heuristic benchmark bot, and the 20-pl
model.py	~80	The dueling network: a convolutional trunk with residual blocks and global pooling, then a value head a
buffer.py	~110	The replay memory and the n-step window that assembles transitions and computes the sign.
train_dqn.py	~330	The training loop: collect, learn, evaluate, checkpoint. Also the command-line interface.
selftest.py	~90	Five correctness checks that run before any training starts.

4.1 One iteration of the loop

1. Collect 8 moves across all 384 games. For each, encode the boards, run one forward pass, mask illegal moves, take the best action or a random one with probability epsilon.

2. Push each move into its game's n-step window. Completed windows become transitions and go into the replay buffer.

3. Do 8 gradient steps (12 for run B). Each samples 512 transitions, rebuilds the features, computes the Double-DQN target, and takes one AdamW step on a Huber loss.

4. Every 2,000 updates, copy the online network into the target network.

5. Every 20 minutes, play 200 games against the fixed bot and record the score.

6. Every 15 minutes, write a checkpoint.

4.2 Why a Huber loss and not squared error

Q targets contain outliers - a position where the bootstrap is badly wrong produces a huge error, and squared error lets that one sample dominate the batch. Huber loss grows linearly past a threshold (beta = 0.1 here), so a bad target moves the weights a little instead of a lot. Gradients are additionally clipped at norm 5.

5. Every hyperparameter, and the reason for it

Setting	Run A	Run B	Why
learning rate	3e-4	1e-3	A is the safe AdamW default for a small conv net; B tests whether faster st
n-step	3	5	Longer windows shorten the credit chain but add variance. The contrast is
epsilon start / end	1.0 / 0.05	1.0 / 0.05	Start fully random, settle at 5% so exploration never fully stops.
epsilon decay	4.0 M steps	1.5 M steps	How long exploration lasts. See the note below - both were too short.
target sync	2,000	1,000	How often the target network is refreshed. Less often = more stable, slowe
updates per burst	8	12	The replay ratio. B trains harder per unit of experience.
discount gamma	1.0	1.0	No discounting. The episode is exactly 60 moves and a box is worth the sa

batch size	512	512	Fills a T4 without thrashing memory.
replay capacity	600,000	600,000	About 10,000 games of history; large enough that the buffer is not domina
min buffer	50,000	50,000	Do not train on an empty buffer.
parallel games	384	384	One forward pass decides 384 moves. This is what makes the GPU worth
network	64 ch x 6	64 ch x 6	Same on both so the comparison is about the learning settings, not capaci
Huber beta	0.1	0.1	Outlier tolerance on Q targets.
grad clip	5.0	5.0	Safety against a bad batch.

6. How to run it

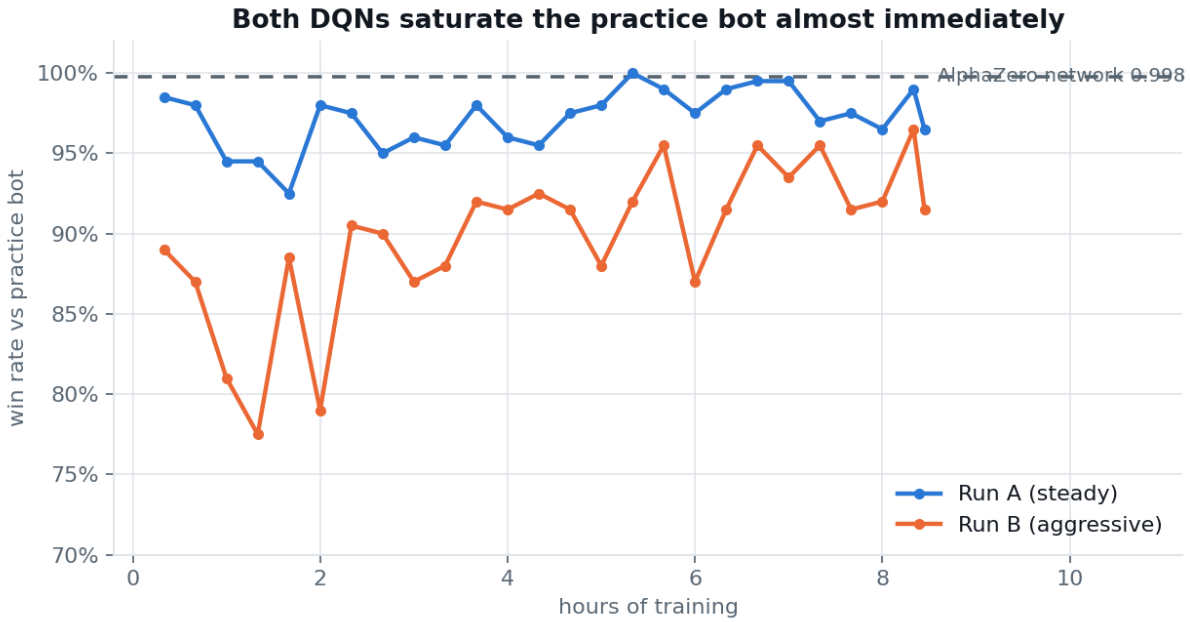
```
python selftest.py # 5 checks, ~20 s, must pass first

python train_dqn.py --board 5 --out runs/dqn_a --hours 8.5 \ --games 384 --channels 64
--blocks 6 --batch 512 \ --lr 3e-4 --nstep 3 --eps-decay-steps 4000000 \ --target-sync
2000 --updates 8 --eval-minutes 20

python train_dqn.py --eval-only --resume-from runs/dqn_a
```

The run is crash-safe: it checkpoints the network, the target network, the optimiser state and the replay buffer every 15 minutes, resumes from where it stopped, and saves on SIGTERM or on finding a file named STOP in its output folder. That is how it survived a full night on Kaggle with the browser closed.

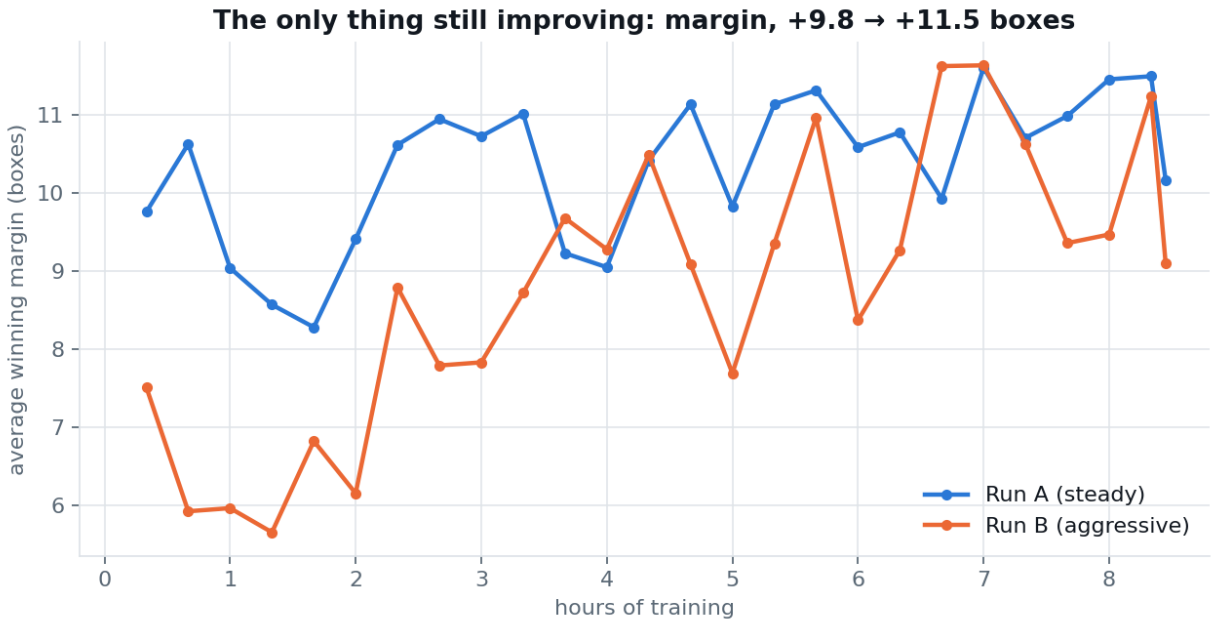
7. What happened during the night



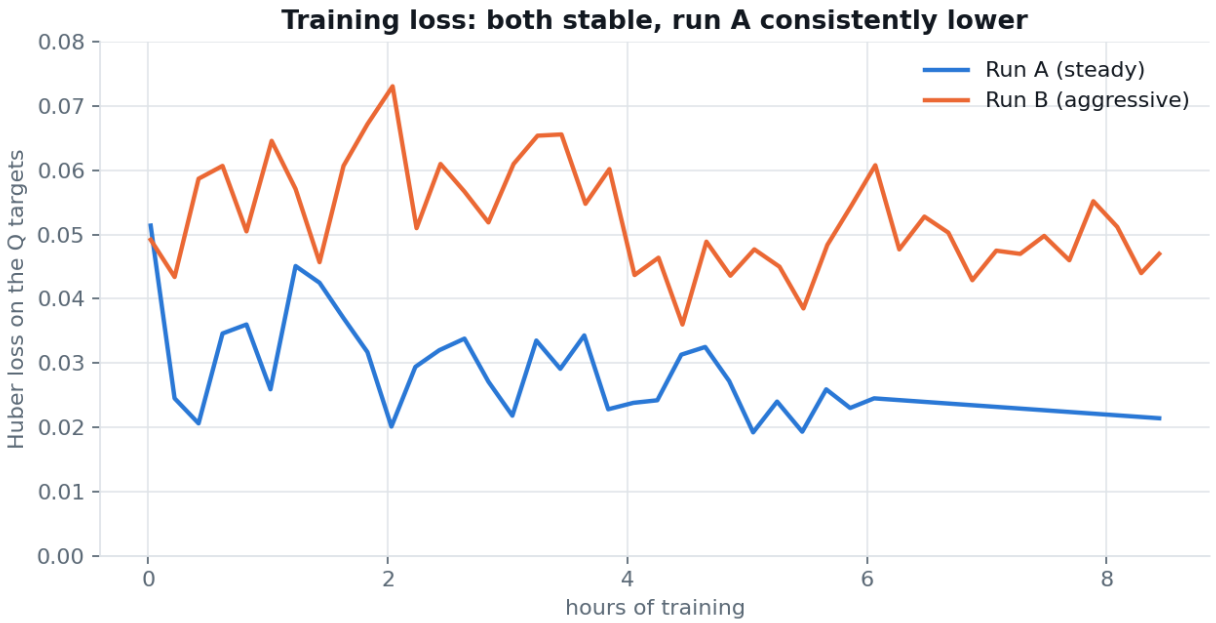
Win rate against the fixed benchmark bot, measured every 20 minutes. Run A reached 98.5% at the first measurement - twenty minutes in - and never meaningfully improved on it. The dashed line is where the AlphaZero network sits on the same test.

This is the most important graph in the report, and not for the reason it looks like. The DQN learned to beat the benchmark bot **within the first twenty minutes**. The remaining eight hours moved the curve almost not at all. That is not a plateau in learning - as the next graph shows, something kept improving. It is a plateau in the *measurement*. The benchmark was too weak to register any further progress.

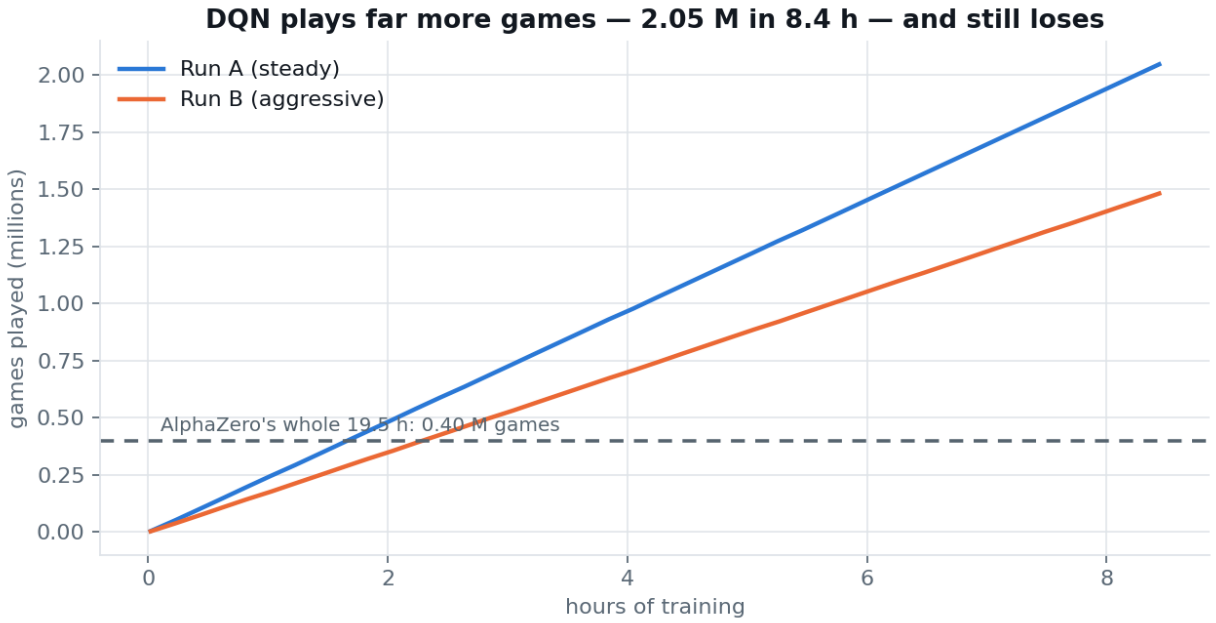
> ***In plain words.*** *It is like testing a student who scores 98% on the first day. Keep giving them the same test for eight hours and you learn nothing about how much better they got.*



Average winning margin over the same games. This kept climbing - roughly +9.8 boxes at the start to +11.5 by the end - which is the evidence that learning continued even after the win rate saturated.



Huber loss on the Q targets. Both runs are stable with no divergence, and run A sits consistently lower - an early hint that A's gentler settings were working better.



Games played. Run A finished 2.05 million games in 8.4 hours. The dashed line is the total for the entire AlphaZero project - 397,000 games over 19.5 hours - which DQN passed inside the first ninety minutes.

A mistake worth recording

Both exploration schedules were sized for a throughput estimate that turned out to be far too low. Run B reached its 5% floor after nine minutes and run A after about twenty, which means both spent roughly 97% of the night at minimum exploration. This was noticed while the run was live and deliberately not corrected, because restarting would have cost the remaining GPU quota. If the runs are repeated, the decay should be set in *hours* rather than in steps, or scaled to measured throughput.

8. Results

Matchup	Games	W-D-L	Score	Margin
AlphaZero network vs practice bot	400	399-0-1	99.8%	+12.41
DQN run A vs practice bot	400	389-0-11	97.3%	+10.48
DQN run B vs practice bot	400	370-0-30	92.5%	+10.04
DQN run A vs DQN run B	200	200-0-0	100.0%	+17.00
DQN run A vs AlphaZero, no search	200	0-0-200	0.0%	-9.00
DQN run A vs full AlphaZero engine	24	0-0-24	0.0%	-10.00

Three things stand out.

The A/B test paid for itself. Against the bot, run A and run B look similar - 97.3% against 92.5%. Played directly against each other, run A won **200 games out of 200** by an average of seventeen boxes. The gentler settings were not slightly better, they were overwhelmingly better, and the benchmark completely failed to show it. Had we run one configuration instead of two, there was a coin-flip chance of shipping the much weaker one and never knowing.

The benchmark was the weak link. AlphaZero scores 99.8% and DQN scores 97.3% on the same test, a gap that looks like noise. Put them on the board together and it is 200-0. Any conclusion drawn from the bot scores alone would have been wrong.

The verdict is not close. With search switched off on both sides - the fairest possible comparison of the two learning algorithms, network against network, no thinking time for either - the DQN lost every game by an average of nine boxes. With AlphaZero's search and exact solver switched back on, it lost every game by ten. 200 losses out of 200 is not a sampling artefact.

9. What this proves, and what it does not

It proves the DQN implementation is correct and competent. It learned a genuinely hard game from scratch, with no human games and no hand-written strategy, to the point of beating a reasonable opponent 97% of the time. Every correctness test passes. This is a working value-based agent, not a broken one.

It proves AlphaZero is far stronger on this game, and that the gap is not about search. That is the finding worth keeping. It would have been easy to assume DQN loses because AlphaZero gets to think and it does not. But with thinking removed from both sides, the gap was still total. AlphaZero's *network* is better than DQN's network, and search is an additional advantage on top.

It does not prove DQN could never get there. Eight hours is one night. Both runs had a badly sized exploration schedule. Prioritised replay, distributional Q-learning, a larger network and a properly tuned schedule are all untried. A fair statement is: given one night each, AlphaZero wins overwhelmingly.

It does not transfer to problems without a simulator. The whole reason AlphaZero wins here is that Dots & Boxes can be simulated perfectly and for free, so search-based improvement costs nothing. On a drone, where the dynamics are approximate at best, that advantage disappears and the model-free methods are the right tool. Same question, opposite answer, and the thing that flips it is whether you own a model you can trust.

10. If this is continued

Replace the benchmark first. Nothing else matters until there is a test that can tell two good agents apart. The obvious choice is already available: score against the AlphaZero network with search off. It is strictly harder than the bot and it does not saturate.

Fix the exploration schedule. Set the decay from measured throughput, or in wall-clock hours. Both runs effectively trained without exploration.

Then change one thing at a time. Prioritised replay is the highest-value single addition for a sparse-credit problem like this. After that, a wider network, since 480,546 parameters is modest for a 60-action game.

Do not expect a different verdict cheaply. The published attempts at Q-learning on this exact game plateau on 3x3 boards; the methods that win competitions on 5x5 are all in the AlphaZero family, and the strongest of them trained for roughly 240 hours. Closing this gap is a multi-week project, not a multi-night one.

Files produced

models/dqn_a.pt	Run A, the stronger model. 480,546 parameters.
models/dqn_b.pt	Run B, kept for comparison.
graphs/*.png	The five figures in this report.
dqn_code/	The five source files, exactly as they ran on Kaggle.
eval_results.json	Raw numbers behind every matchup in section 8.
history.json	The full training history used to draw the graphs.